Universidad del Rosario | Escuela de Ciencias e Ingeniería | DEPE

# INTRODUCCIÓN A PYTHON: FUNDAMENTOS DEL LENGUAJE

SÁBADO
**16 DE AGOSTO** | **10:00 AM A 12:00 PM**

En esta sesión de dos horas, exploraremos los conceptos esenciales de Python, uno de los lenguajes de programación más populares y versátiles del mundo.

## Taller Knuth

**MODALIDAD ACCESO REMOTO**

NO SE REQUIERE EXPERIENCIA PREVIA EN PROGRAMACIÓN

# Bienvenida

**Sobre el conferencista**



**Miguel Angel Orjuela Rocha**

Ing. de Sistemas y Computación

**Skills:**

- Desarrollo Fullstack
- Analítica de datos
- Transferencia de conocimiento

**Contacto:**


miguela.orjuela@urosario.edu.co

https://www.linkedin.com/in/miguel-orjuela/

https://github.com/maorjuela73

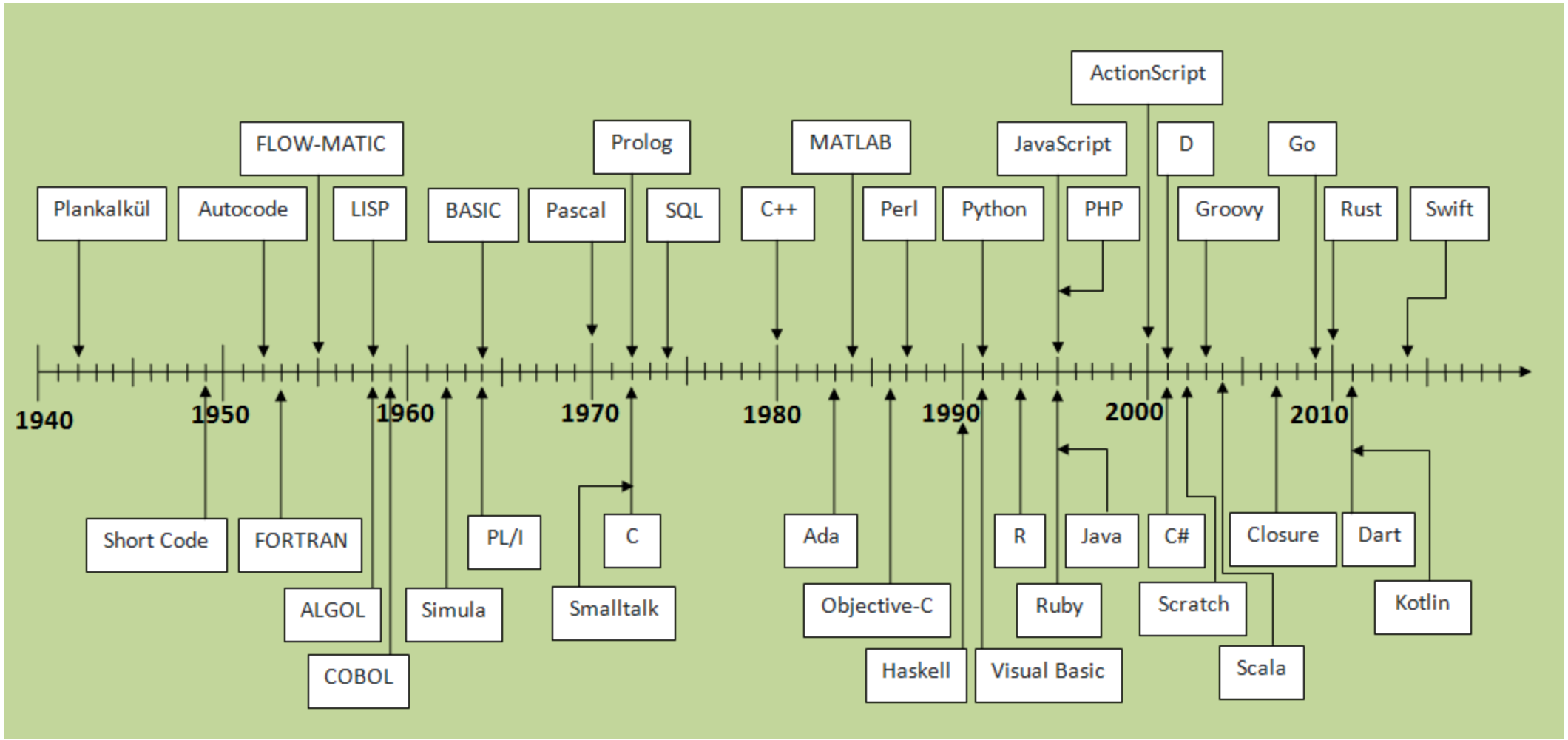
# Contenido

- ¿Qué es Python?
- Instalación
- Ejecución de código
- Variables, expresiones y operaciones básicas
- Indentación
- Ejecuciones condicionales y cíclicas
- Algunos retos

# ¿Qué es Python?

# Historia de los lenguajes de programación



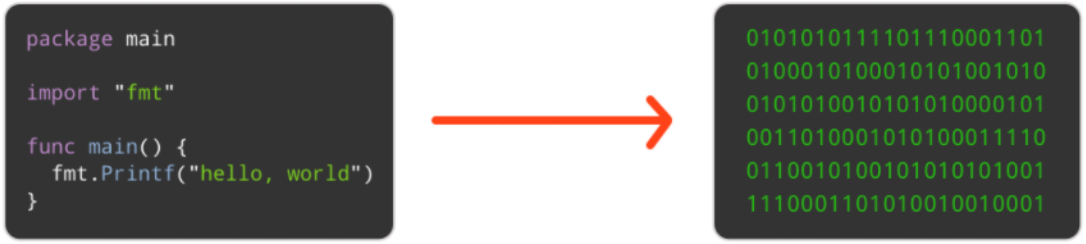



https://javaconceptoftheday.com/history-of-programming-languages/

# Características de Python



- Lenguaje de programación **interpretado** de alto nivel




Lenguaje de alto nivel que entiende el programador

Lenguaje de máquina que entiende el procesador

https://blog.makeitreal.camp/lenguajes-compilados-e-interpretados

- Multipropósito
  - Data analytics/Big data/Data science/IA/Blockchain
  - Desarrollo web
  - Desarrollo de videojuegos
  - Glue code

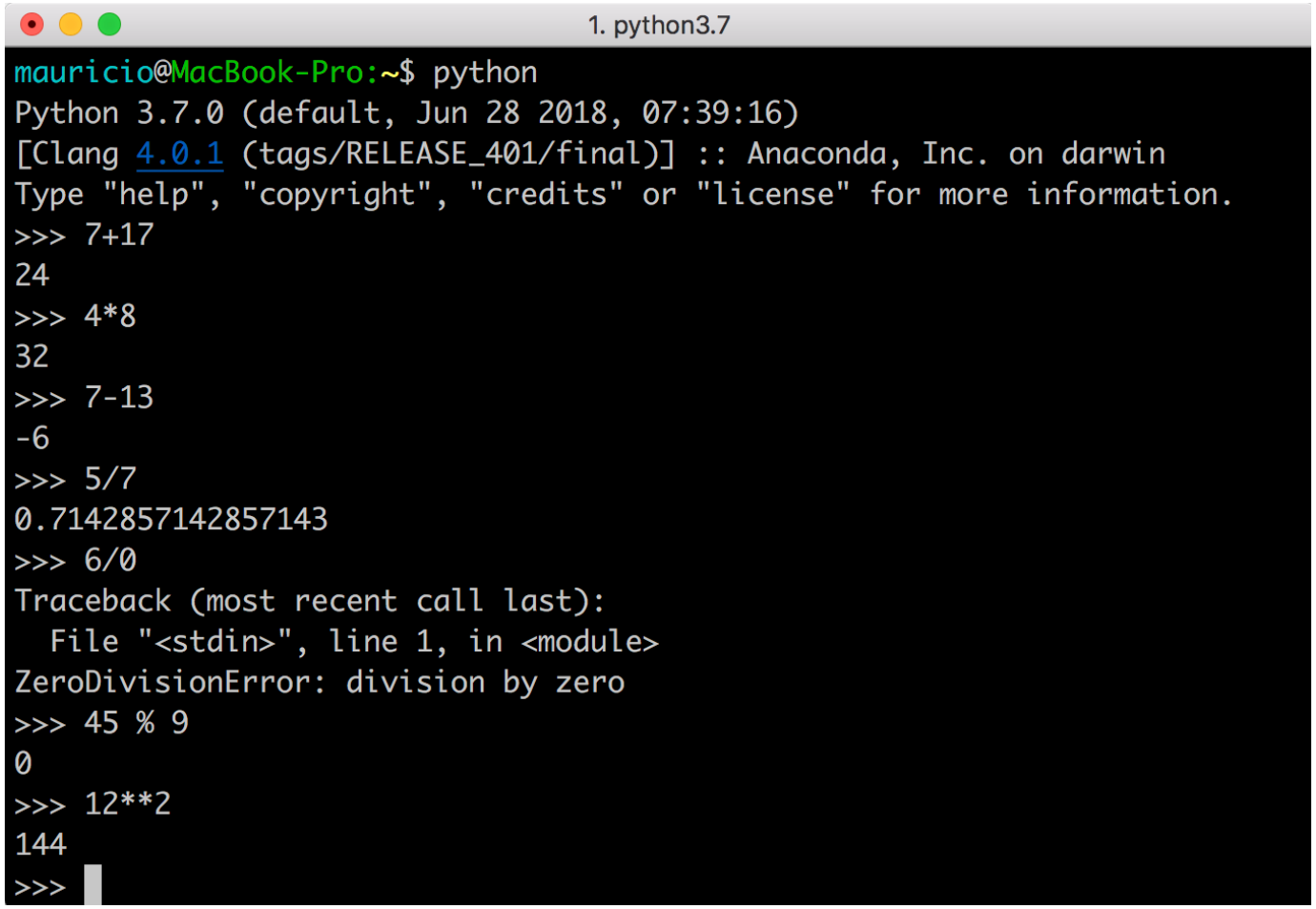
# Instalación

# Python y Entornos de Desarrollo



**Intérprete** de Python

- https://www.python.org/



# Python y Entornos de Desarrollo

**Instalación**

⚙️ Un **IDE (Integrated Development Environment)** proporciona herramientas para escribir, depurar y ejecutar código de manera eficiente.

🔷 **Principales IDEs para Python**:

🐍📎 **Spyder** – Ideal para ciencia de datos y análisis numérico.
📎 https://www.spyder-ide.org/

📜📎 **Jupyter Notebook** – Para ejecutar código en celdas interactivas.
https://jupyter.org/try

🔥📎 **PyCharm** – IDE profesional con funciones avanzadas.
📎 https://www.jetbrains.com/es-es/pycharm/

💻 **Visual Studio Code** – Editor ligero con excelente integración para Python.
📎 https://code.visualstudio.com/

✏️📎 **Sublime Text** – Rápido y personalizable.
📎 https://www.sublimetext.com/



# Python y Entornos de Desarrollo



## 🌐 Si prefieres no instalar nada en tu PC, usa entornos en la nube

🔹 **Principales plataformas**:

🔬 **Google Colab** – Ejecuta código en la nube con acceso a GPUs.
https://colab.research.google.com/

🚀 **Replit** – Plataforma colaborativa para escr
https://replit.com/



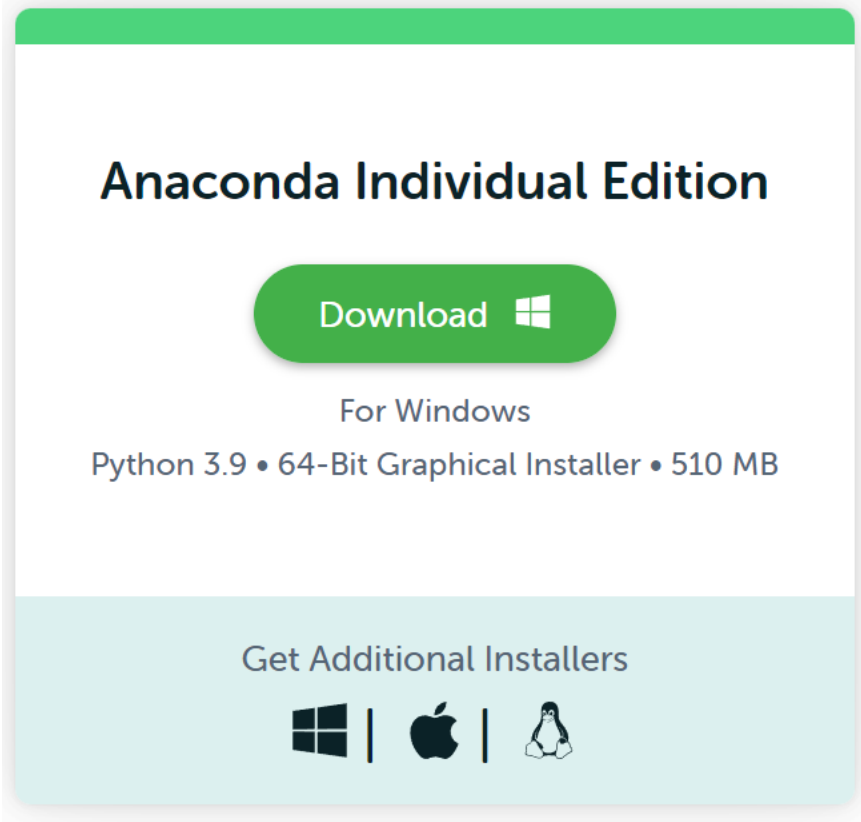
# Anaconda

**Instalación**

**Intérprete** de Python

+

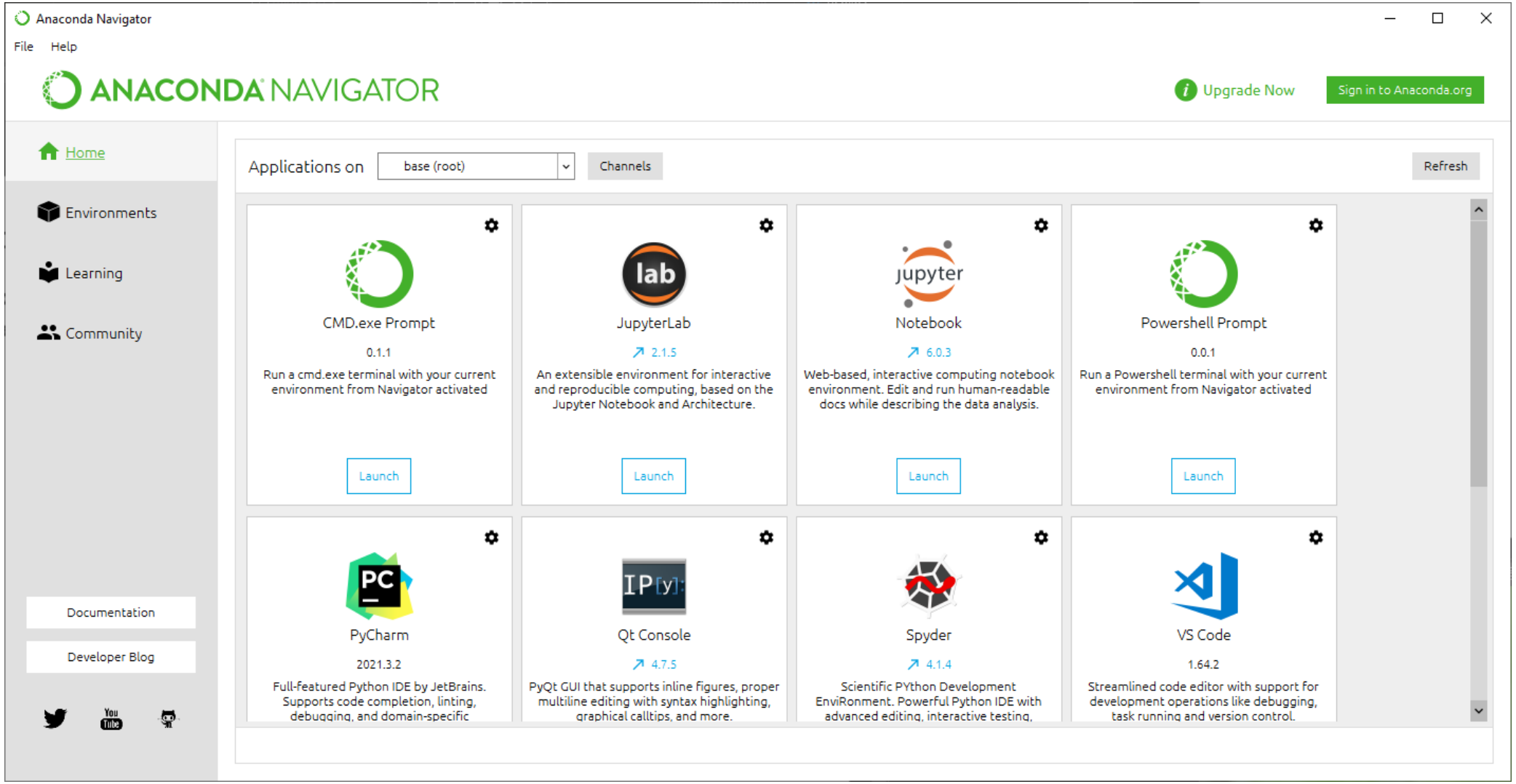
Entornos Integrados de Desarrollo (**IDEs**)




https://www.anaconda.com/

# Anaconda

Instalación



# Ejecución de código

# Directamente en el intérprete



1. Asegurarse de tener el Python instalado
2. Abrir una terminal de línea de comandos




En Windows



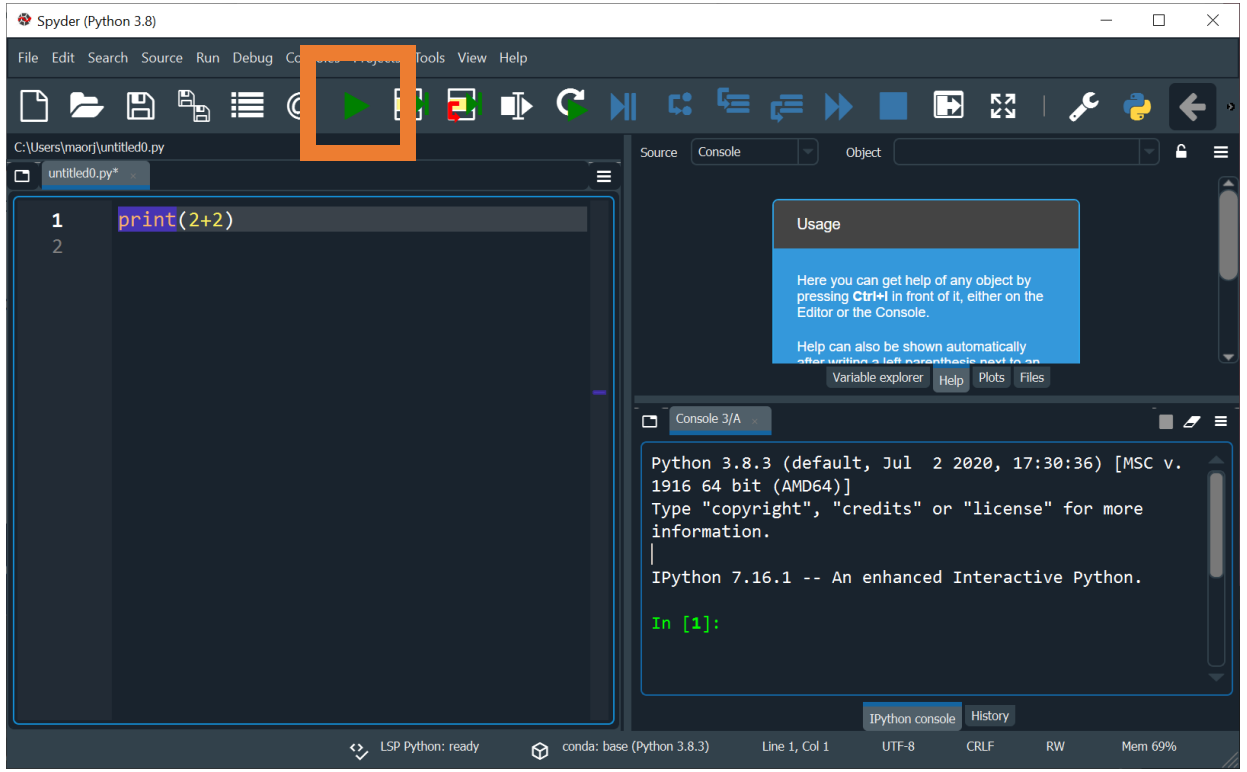

En MacOS

3. Ejecutar el comando **`python`** o **`python3`**
4. Salir con el comando **`exit()`**

# En spyder



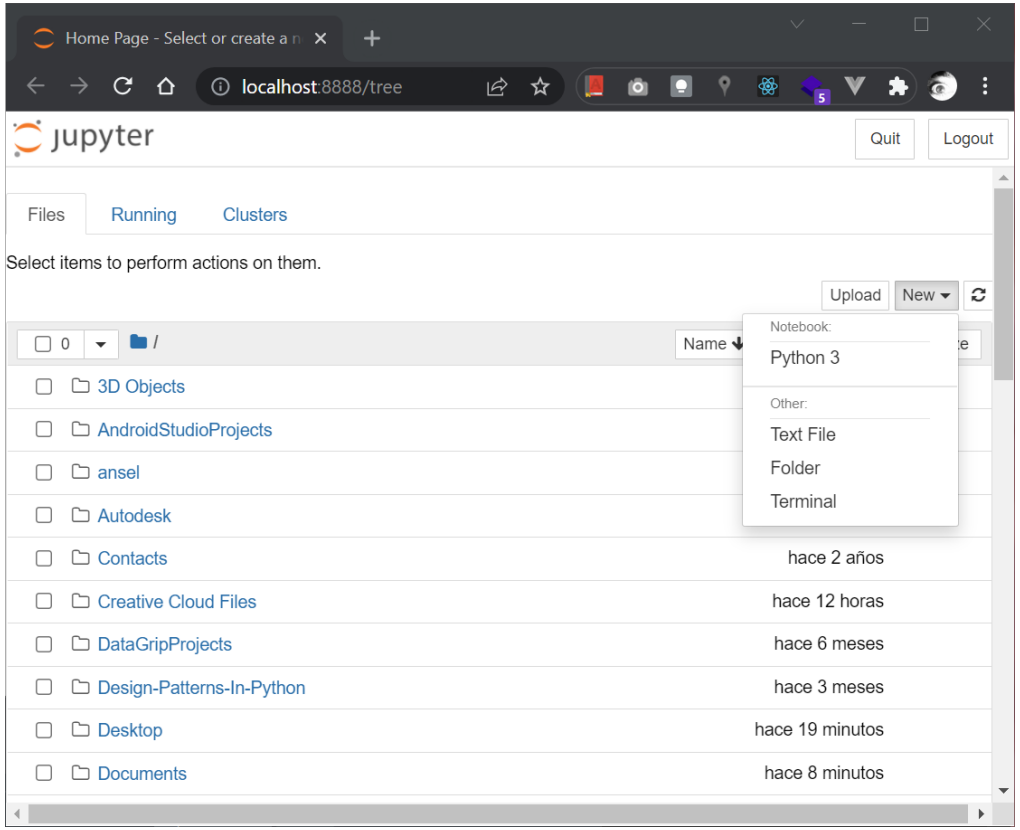
- Asegurarse de tener el Python instalado
- Dar en el botón Play

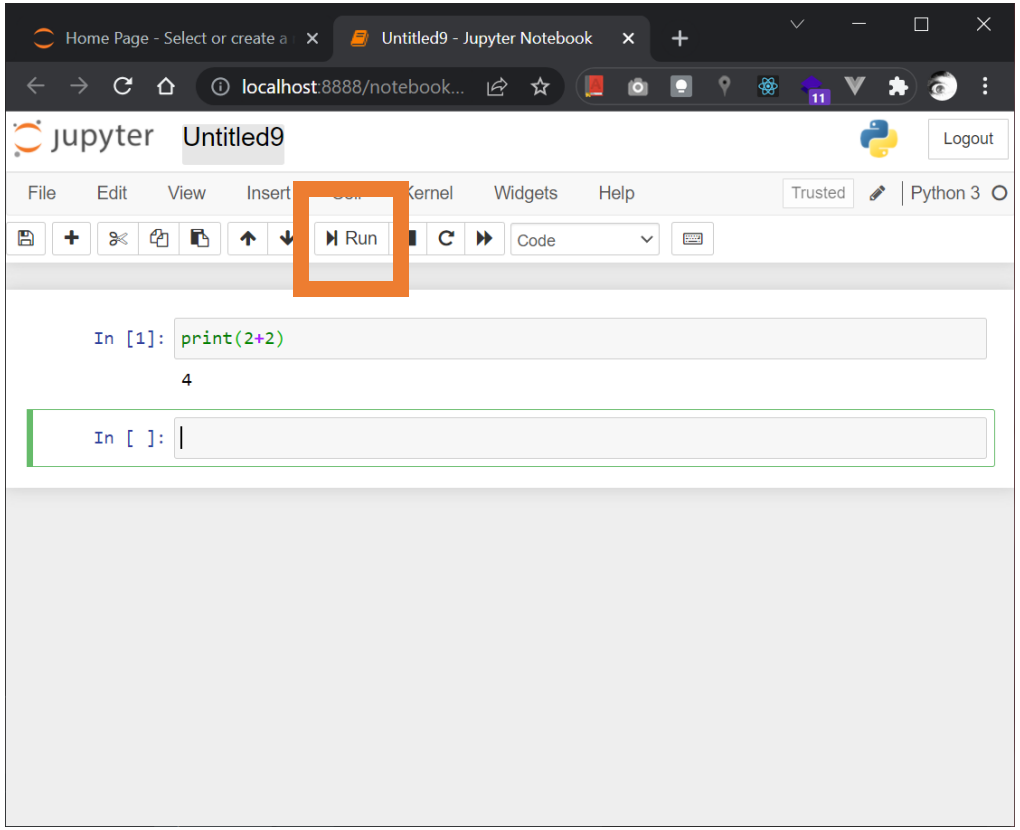


# En Jupyter

## Ejecución de código

- Asegurarse de tener el Python instalado
- Dar en el botón Play

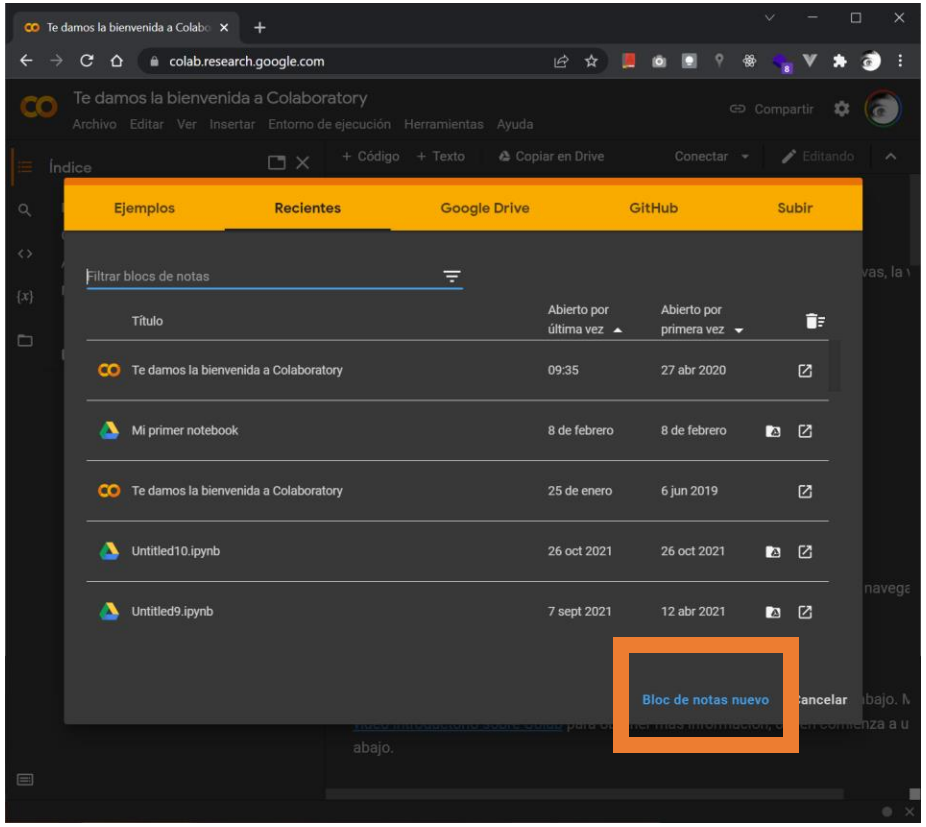


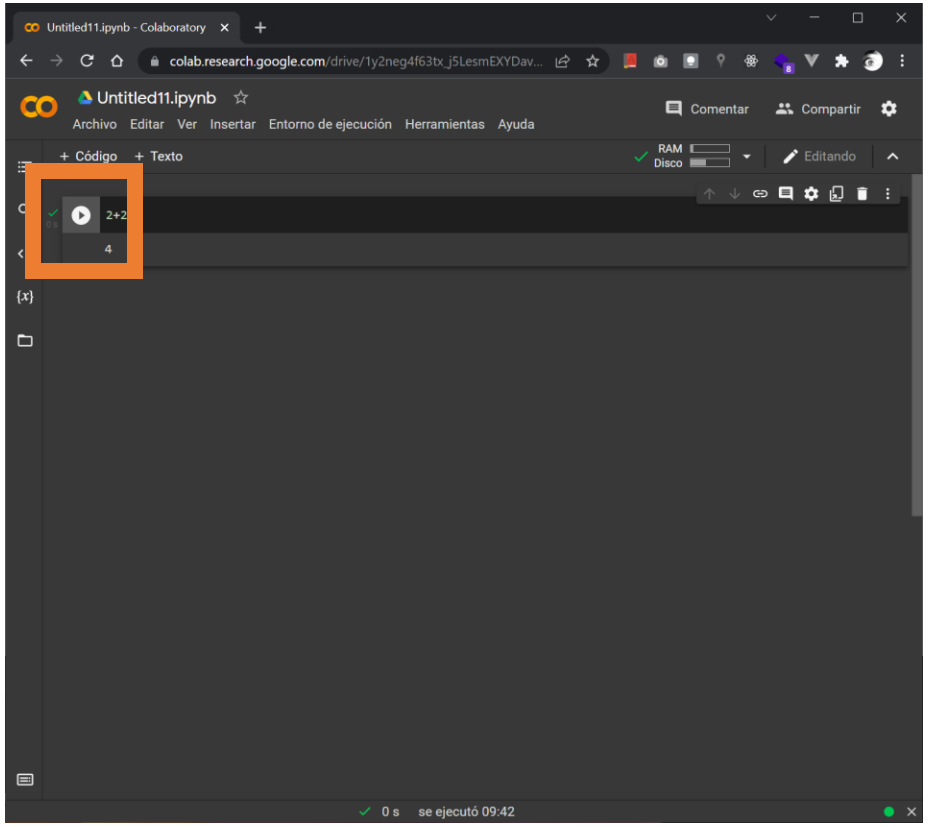



# En Colab



- Igual que Jupyter



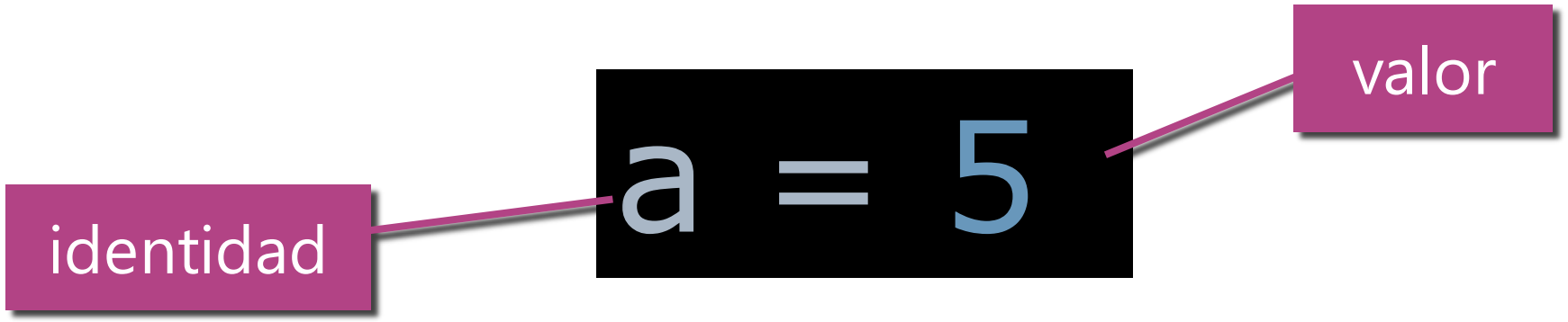
# Variables, expresiones y operaciones básicas

# Objetos en Python



- Los objetos son la abstracción de Python para los datos.
- Todos los datos en un programa de Python están representados por **objetos** o por **relaciones entre objetos**.

Cada objeto tiene una *identidad*, un *tipo* y un *valor* asociado



# Crear variables



- Cada variable tiene un tipo de datos adecuado para el dato que representa

```
mensaje = "Hola mundo"
print (mensaje)
print(type(mensaje))

n = 10
print(n)
print(type(n))

pi = 3.141592
print(pi)
print(type(pi))
```

- Es posible convertir un dato a otro tipo

```
int(pi)
float(15)
float("2.7182")
```

# ¿Qué es una expresión?

Combinación de constantes, variables o funciones, que es interpretada de acuerdo a la síntaxis de un lenguaje particular

```
# Esto es un comentario
m = 2
b = 3.5
x = 3
y = (m*x)+b
print(y)
```

```
'''
Este es un bloque
de comentarios
'''
```

# Operadores

**Expresiones**

Aritméticos

| OPERATOR | DESCRIPTION | SYNTAX |
|---|---|---|
| + | Addition: adds two operands | x + y |
| - | Subtraction: subtracts two operands | x - y |
| * | Multiplication: multiplies two operands | x * y |
| / | Division (float): divides the first operand by the second | x / y |
| // | Division (floor): divides the first operand by the second | x // y |
| % | Modulus: returns the remainder when first operand is divided by the second | x % y |

```
a = 3 + 2
b = 5 - 3
c = 4 * 8
d = 7 / 4
e = 7 // 4
f = 7 % 4
g = 3 ** 4
```

# Entrada de datos



```
n = input("Digite un numero ")
print(type(n))
print("El numero es:", n)
```

# Expresiones booleanas y valores lógicos



## Operadores relacionales

| OPERATOR | DESCRIPTION | SYNTAX |
|---|---|---|
| > | Greater than: True if left operand is greater than the right | x > y |
| < | Less than: True if left operand is less than the right | x < y |
| == | Equal to: True if both operands are equal | x == y |
| != | Not equal to - True if operands are not equal | x != y |
| >= | Greater than or equal to: True if left operand is greater than or equal to the right | x >= y |
| <= | Less than or equal to: True if left operand is less than or equal to the right | x <= y |

## Operadores lógicos

| OPERATOR | DESCRIPTION | SYNTAX |
|---|---|---|
| and | Logical AND: True if both the operands are true | x and y |
| or | Logical OR: True if either of the operands is true | x or y |
| not | Logical NOT: True if operand is false | not x |

```
x = 10, y = 20, z = 30
x < y and z % 2 == 0
x % y == 0 or z < y
```

| A | B | Y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

Operador **Y**

| A | B | Y |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

Operador **Ó**

| A | B |
|---|---|
| 0 | 1 |
| 1 | 0 |

Operador **NO**

# Indentación

# Estructura de bloques

**Indentación**

- Python utiliza la indentación para delimitar la estructura permitiendo establecer bloques de código.
- No existen comandos para finalizar las líneas ni llaves con las que delimitar el código.
- Los únicos delimitadores existentes son los dos puntos ( **:** ) y la indentación del código.

```
edad = int(input("Digite su edad: "))
if edad <= 18:
    print("Usted es menor de edad")
elif edad > 18 and edad < 60:
    print("Usted es mayor de edad")
else:
    print("Usted es adulto mayor")
```

# Indentación



- Son tabulaciones, o espacios (sangrías)
- Las ejecuciones de código son interpretadas siguiendo un orden de indentación.

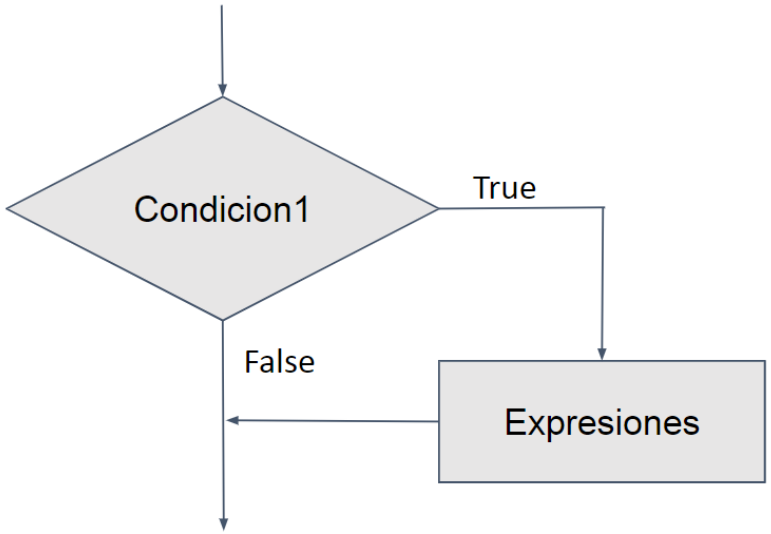


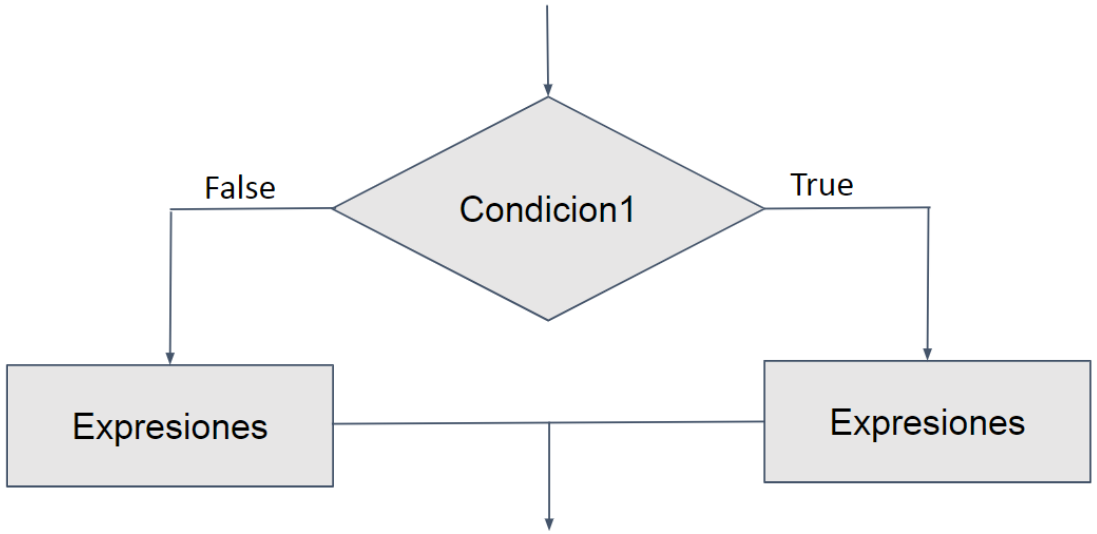

```
edad = int(input("Digite su edad: "))
if edad <= 18:
    print("Usted es menor de edad")
elif edad > 18 and edad < 60:
    print("Usted es mayor de edad")
else:
    print("Usted es adulto mayor")
```

# Ejecuciones condicionales y cíclicas

# Ejecución condicional

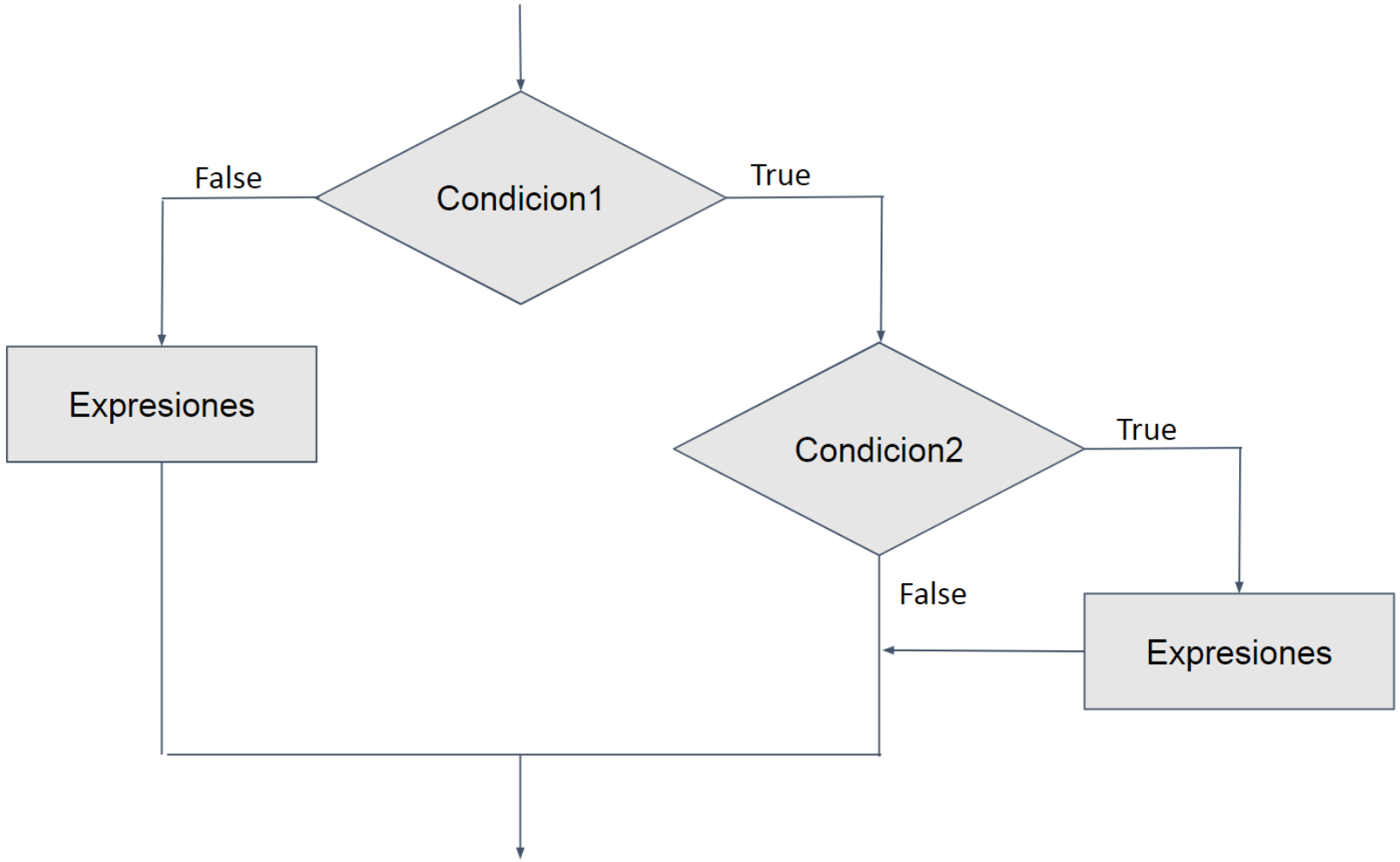




# Ejecución condicional





# Ejemplos: Ejecución condicional



```
x = int(input("Digite numero: "))
if x % 2 == 0:
    print(x, " es par.")
else:
    print(x, " es impar.")

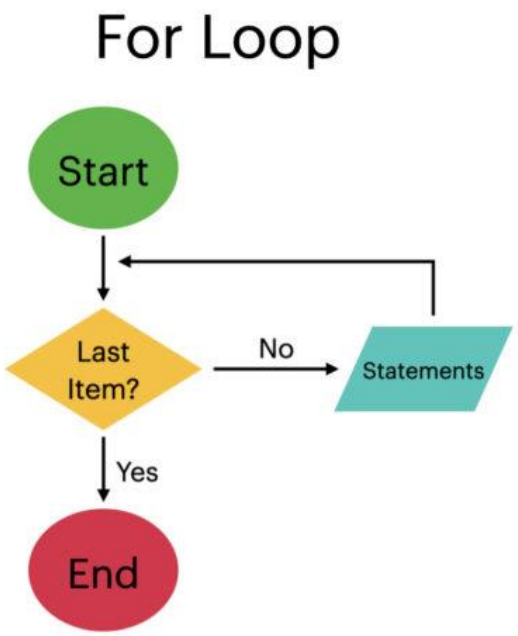


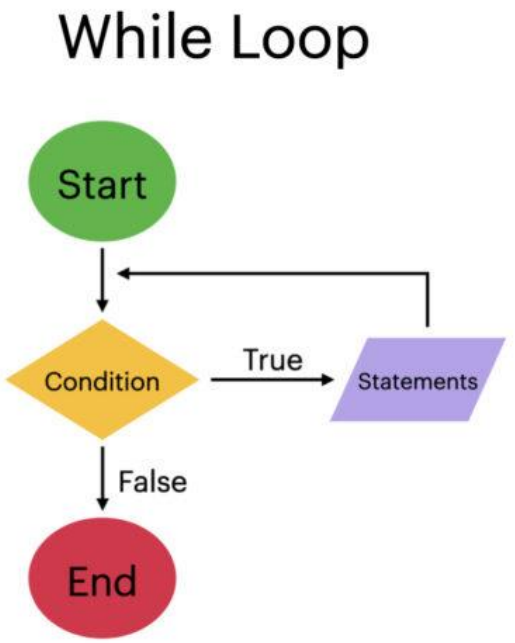
x = int(input("Digite numero: "))
if x % 2 == 0:
    print(x, " es divisible por 2.")
elif x % 3 == 0:
    print(x, " es divisible por 3.")
else:
    print(x, " no es divisible por 2 ni por 3.")
```

```
edad = int(input("Digite su edad: "))
if edad <= 18:
    print("Usted es menor de edad")
elif edad > 18 and edad < 60:
    print("Usted es mayor de edad")
else:
    print("Usted es adulto mayor")
```

# Ejecución cíclica





# Ejemplos: Ejecución cíclica



```
for i in range(0, 10):
    print (i)

for i in range(10):
    print (i)

for i in range(0, 10, 2):
    print (i)

lista = [10, 20, 30, 40]
for i in lista:
    print (i)
```

```
i = 1
while i < 6:
  print(i)
  i += 1
```

# Algunos retos

# Reto # 1

- Imprimir la sumatoria de todos los múltiplos de 3 o 5 menores a 1000

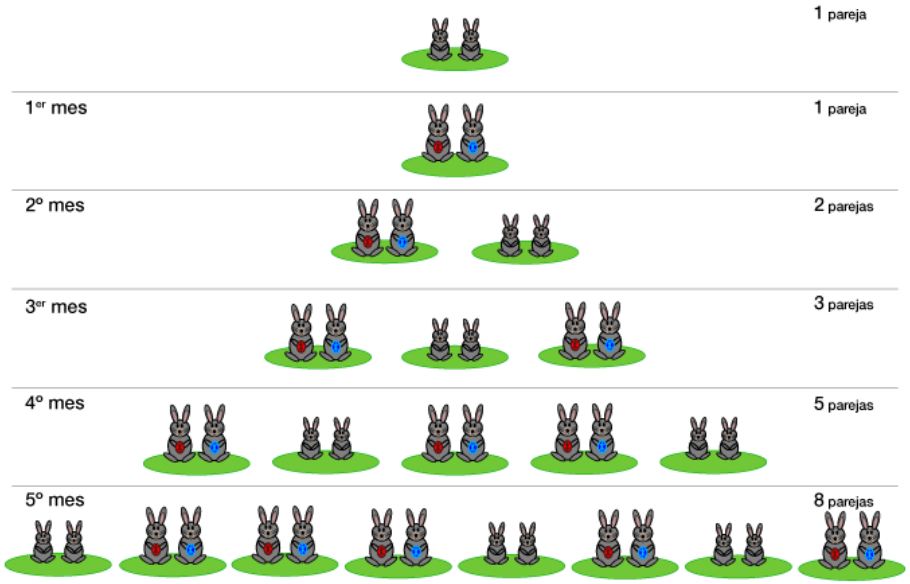
# Reto # 2

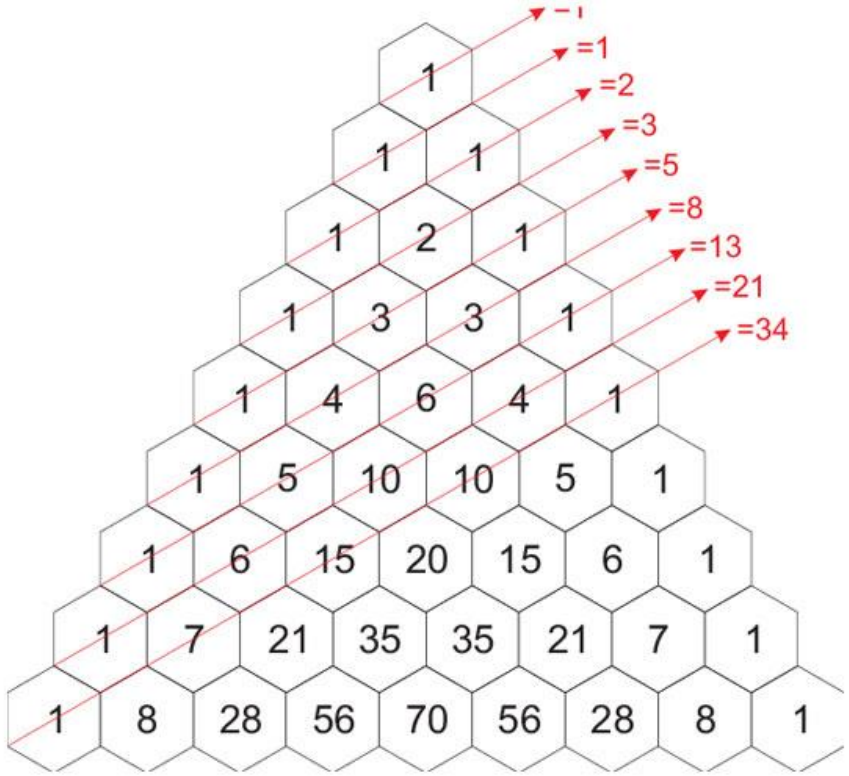
- Imprimir los números primos menores que 100

| 0 | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
| 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
| 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 |
| 31 | 32 | 33 | 34 | 35 | 36 | 37 | 38 | 39 | 40 |
| 41 | 42 | 43 | 44 | 45 | 46 | 47 | 48 | 49 | 50 |
| 51 | 52 | 53 | 54 | 55 | 56 | 57 | 58 | 59 | 60 |
| 61 | 62 | 63 | 64 | 65 | 66 | 67 | 68 | 69 | 70 |
| 71 | 72 | 73 | 74 | 75 | 76 | 77 | 78 | 79 | 80 |
| 81 | 82 | 83 | 84 | 85 | 86 | 87 | 88 | 89 | 90 |
| 91 | 92 | 93 | 94 | 95 | 96 | 97 | 98 | 99 | 100 |

# Reto # 3

- En la serie de Fibonacci, cada término se calcula sumando los dos términos anteriores. Considerando los 20 primeros términos de la serie, cuántos términos son pares y cuántos son impares?



# Gracias por tu asistencia y participación ☺

**Contacto**

miguela.orjuela@urosario.edu.co

https://www.linkedin.com/in/miguel-orjuela/

https://github.com/maorjuela73

**Links de interés**

- *https://learnbyexample.gitbooks.io/python-basics/content/*
- *https://www.rstudio.com/resources/cheatsheets/*